



Partie I

Préparation TP2

Programmation des Microcontrôleurs



- ✚ **Support du cours : Microcontrôleur ATmega328**

- ✚ **Les Interruptions matérielles : interruptions externes.**
 - ✓ Registres et configuration
 - ✓ Exemples de mise en œuvre

- ✚ **Les bibliothèques (librairie) pour périphériques complexes**
 - ✓ Installation
 - ✓ Essai du périphérique avec exemples fournis par le constructeur
 - ✓ API détaillée produite par le constructeur du périphérique

Rappel:

CC1 le 12/11/26 14:30-16h évaluation écrite

CC2 du 12/11/26 au 19/11/26 TP d'évaluation



Interruptions externes de l'ATMega328

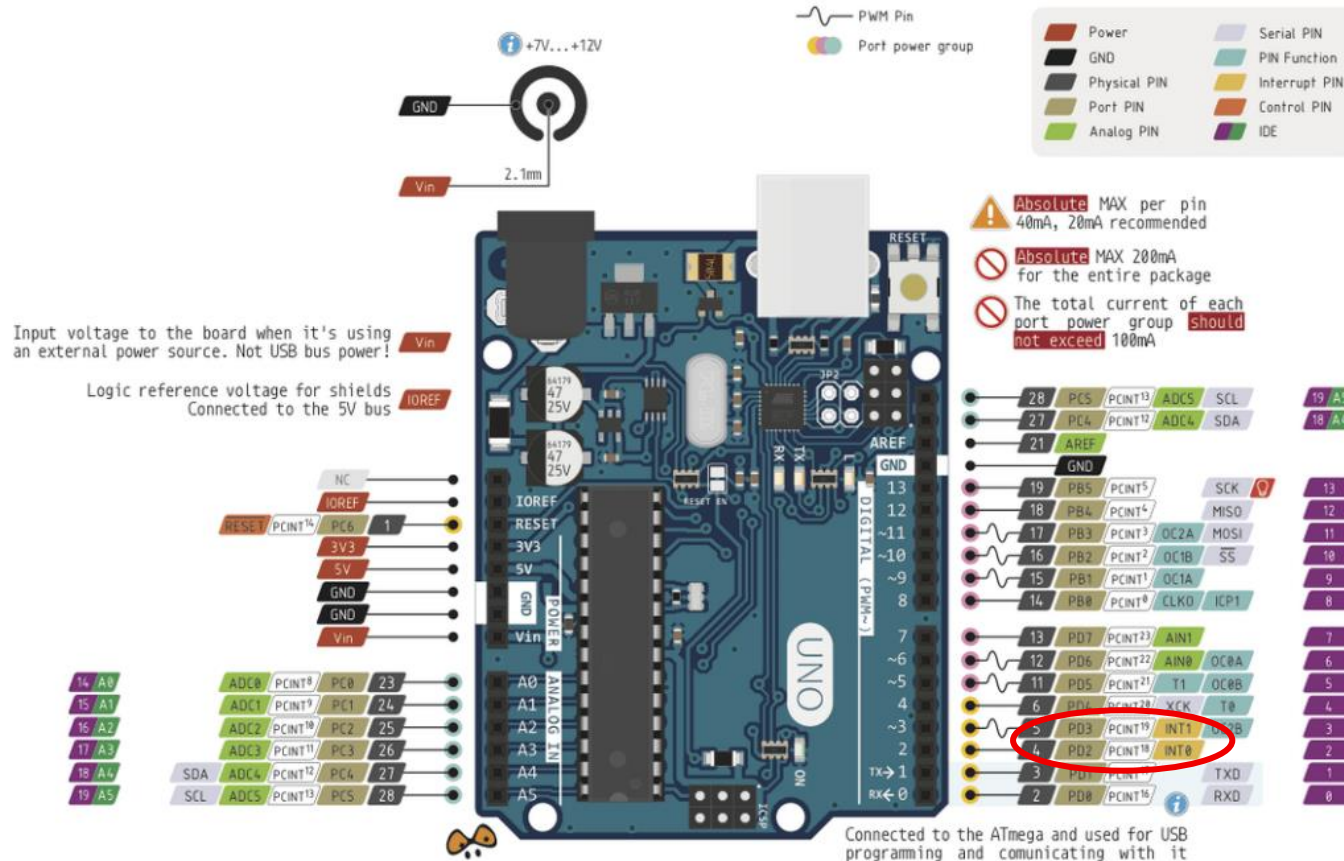


Contraintes matérielles:

Broches utilisables pour les interruptions externes

Sur une Arduino Uno basée sur l'ATmega328P, les interruptions externes peuvent être utilisées sur les broches numériques :

- D2 → interruption INT0
- D3 → interruption INT1





Modes de déclenchement d'une interruption / Entrées TOR

Une interruption peut être déclenchée selon différentes conditions appliquées au signal d'entrée. Le déclenchement peut avoir lieu lors d'un changement d'état (front) ou lorsqu'un niveau logique particulier est détecté.

L'illustration présente les principaux modes de déclenchement : front montant, front descendant, niveau haut, niveau bas et changement d'état.



Rising Edge Trigger: used to detect rising edges without jitter.



Falling Edge Trigger: used to detect falling edges without jitter.



High Level Trigger: used to detect high level.



Low Level Trigger: used to detect low level.



Bilateral Edge Trigger: used to detect bilateral edges without jitter.

API Arduino:

API / fonction	Rôle	Syntaxe générique	Exemple
<code>attachInterrupt()</code>	Associer une fonction à une interruption externe	<code>attachInterrupt(broche, ISR, mode);</code>	<code>attachInterrupt(26, ISR_bouton, RISING);</code>
<code>detachInterrupt()</code>	Désactiver une interruption	<code>detachInterrupt(broche);</code>	<code>detachInterrupt(26);</code>
<code>digitalPinToInterrupt()</code>	Obtenir l'interruption associée à une broche	<code>digitalPinToInterrupt(broche)</code>	<code>digitalPinToInterrupt(26)</code>
RISING	Déclenchement sur front montant	<code>..., RISING);</code>	<code>attachInterrupt(26, ISR, RISING);</code>
FALLING	Déclenchement sur front descendant	<code>..., FALLING);</code>	<code>attachInterrupt(26, ISR, FALLING);</code>
CHANGE	Déclenchement sur changement d'état	<code>..., CHANGE);</code>	<code>attachInterrupt(26, ISR, CHANGE);</code>
LOW	Déclenchement lorsque l'entrée est à l'état bas	<code>..., LOW);</code>	<code>attachInterrupt(26, ISR, LOW);</code>
HIGH	Déclenchement lorsque l'entrée est à l'état haut	<code>..., HIGH);</code>	<code>attachInterrupt(26, ISR, HIGH);</code>
<code>volatile</code>	Variable partagée avec la fonction d'interruption	<code>volatile type variable;</code>	<code>volatile bool etat = false;</code>



Sources d'interruption exploitables sur ATMega328 (carte Arduino UNO)

Définition de base.

Une interruption suspend(interrompt) le programme en cours pour exécuter un traitement particulier .

*C'est différent d'une fonction car n'est pas appelé explicitement par le programme (on n'écrit pas l'appel de ce traitement dans le code source).
C'est le processeur qui, suite à la détection d'une cause particulière, déclenche le traitement de l'interruption. Si elles sont autorisées...!*

- ✓ La « fonction » associée à une interruption est appelée fonction d'interruption ou routine d'interruption : **Interrupt Service Routine (ISR)**.
- ✓ Les périphériques d'E/S d'un microcontrôleur peuvent conduire à déclencher des interruptions. On parlera alors d'interruption matérielle (**Interrupt ReQuest: IRQ**).
- ✓ Ce mécanisme permet d'assurer des temps de réponse très courts entre la cause de l'interruption et son traitement.



Important : Les appels des **ISR** s'insèrent de façon **asynchrone** dans l'exécution du programme car on ne sait pas à l'avance quand elles vont être appelées.

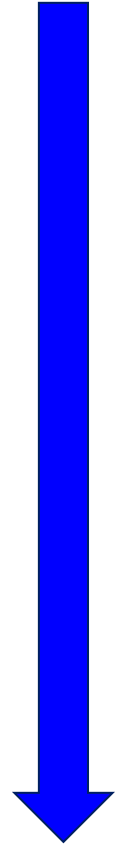
Par exemple, un programme Arduino est interrompu chaque milliseconde par une **ISR** liée au **Timer0**. (Ce n'est pas une **IRQ** !)

Cette routine « **ISR Timer0** » met à jour les variables de temps utilisées par les fonctions **delay()** , **millis()** etc, l'utilisant.

Périodiquement, un programme Arduino est donc suspendu (interrompu) par l'**ISR Timer0**, ce qui ralentit son exécution d'environ 6%.

**Vecteurs d'interruptions, i.e toutes les sources (causes possibles)****11.4 Interrupt Vectors in ATmega328P****Table 11-6. Reset and Interrupt Vectors in ATmega328P**

VectorNo.	Program Address ⁽²⁾	Source	Interrupt Definition
1	0x0000 ⁽¹⁾	RESET	External Pin, Power-on Reset, Brown-out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	PCINT0	Pin Change Interrupt Request 0
5	0x0008	PCINT1	Pin Change Interrupt Request 1
6	0x000A	PCINT2	Pin Change Interrupt Request 2
7	0x000C	WDT	Watchdog Time-out Interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 Compare Match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 Compare Match B
10	0x0012	TIMER2 OVF	Timer/Counter2 Overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 Capture Event
12	0x0016	TIMER1 COMPA	Timer/Counter1 Compare Match A
13	0x0018	TIMER1 COMPB	Timer/Counter1 Compare Match B
14	0x001A	TIMER1 OVF	Timer/Counter1 Overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 Compare Match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 Compare Match B
17	0x0020	TIMER0 OVF	Timer/Counter0 Overflow
18	0x0022	SPI, STC	SPI Serial Transfer Complete
19	0x0024	USART, RX	USART Rx Complete
20	0x0026	USART, UDRE	USART, Data Register Empty
21	0x0028	USART, TX	USART, Tx Complete
22	0x002A	ADC	ADC Conversion Complete

**Plus
prioritaire****Moins
prioritaire**

Exemple simple:

On souhaite changer l'état (Allumée/Eteinte) de la LED embarquée sur la carte Arduino UNO (sur pin 13) lorsque la broche 3(INT1=PD3) passe d'un NLB(Low) à un NLH(High)

```
const byte LED = 8;
const byte interruptPin = 3; //PD3=INT1
volatile byte EtatLED = LOW; // LED Off
```

```
/* ISR qui change l'état de la LED */
```

```
void monISR()
```

```
{
  EtatLED = !EtatLED;
}
```

Code ISR
très court!

```
void setup()
```

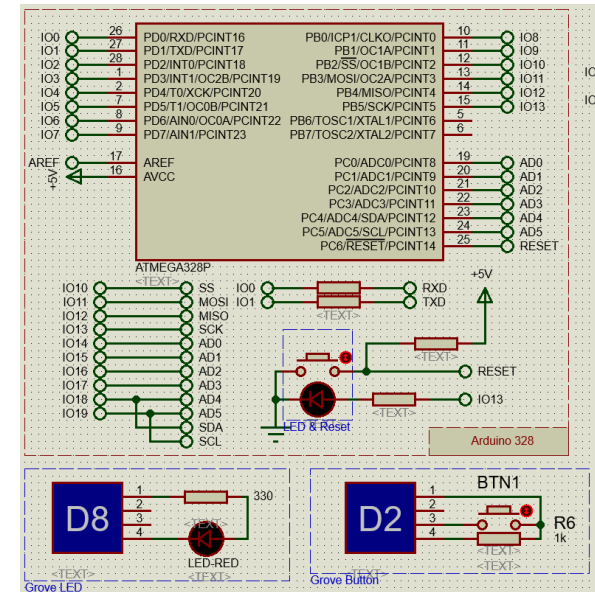
```
{
  pinMode(LED, OUTPUT);
```

```
  pinMode(interruptPin, INPUT_PULLUP);
```

```
  attachInterrupt(digitalPinToInterrupt(interruptPin), monISR,
  CHANGE);
}
```

```
void loop()
```

```
{
  digitalWrite(ledPin, EtatLED); //Pas de digitalRead(BP)!
}
```



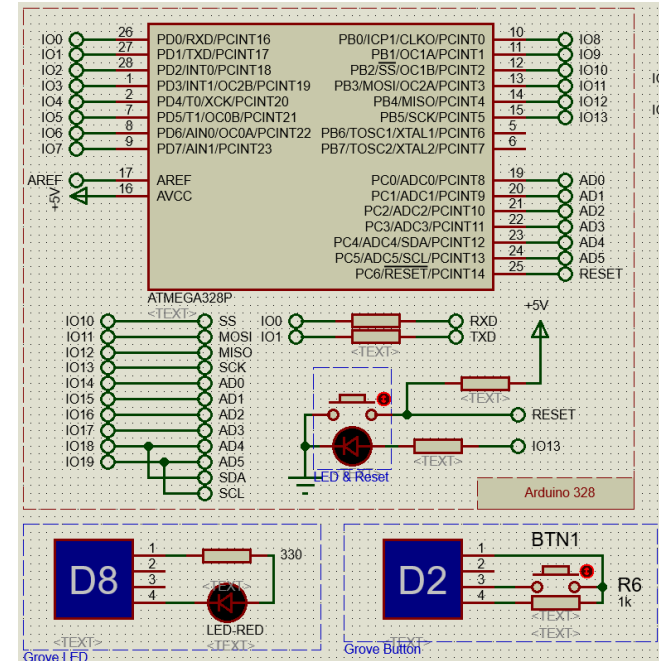
TD interruption externe:

```

const int buttonPin = ____;
const int ledPin = ____;
volatile bool eventButton = ____;
void buttonISR()
{
    eventButton = ____;
}
void setup()
{
    pinMode(buttonPin, ____);
    pinMode(ledPin, ____);
    attachInterrupt(____,
        digitalPinToInterrupt(buttonPin),
        ____);
}
void loop()
{
    if (____)
    {
        eventButton = ____;
        digitalWrite(
            ledPin,
            !digitalRead(ledPin)
        );
    }
}

```

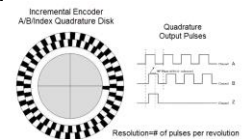
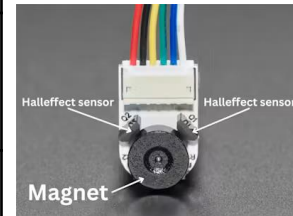
Code ISR
très court!





Encodeur incrémental et interruptions : Contextes d'application

N°	Contexte d'application	Utilisation de l'encodeur	Grandeur mesurée / déterminée
1	Position d'un moteur	Comptage des impulsions lors de la rotation de l'axe	Position angulaire
2	Vitesse d'un moteur	Comptage du nombre d'impulsions pendant une durée donnée	Vitesse de rotation
3	Sens de rotation d'un moteur	Comparaison du déphasage entre les voies A et B	Sens horaire / antihoraire
4	Ascenseur	Mesure de la rotation de la poulie	Position de la cabine
5	Convoyeur industriel	Mesure de la rotation d'un rouleau	Déplacement / vitesse
6	Robot mobile	Encodeur placé sur les roues	Distance parcourue / vitesse

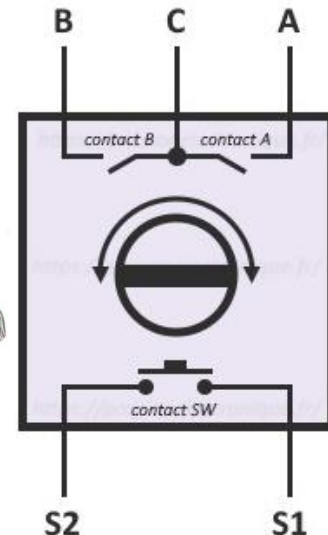
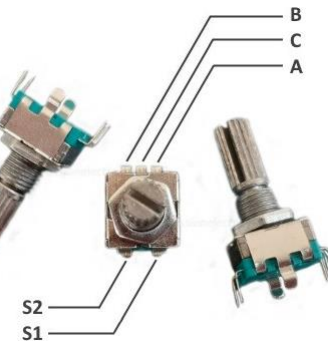
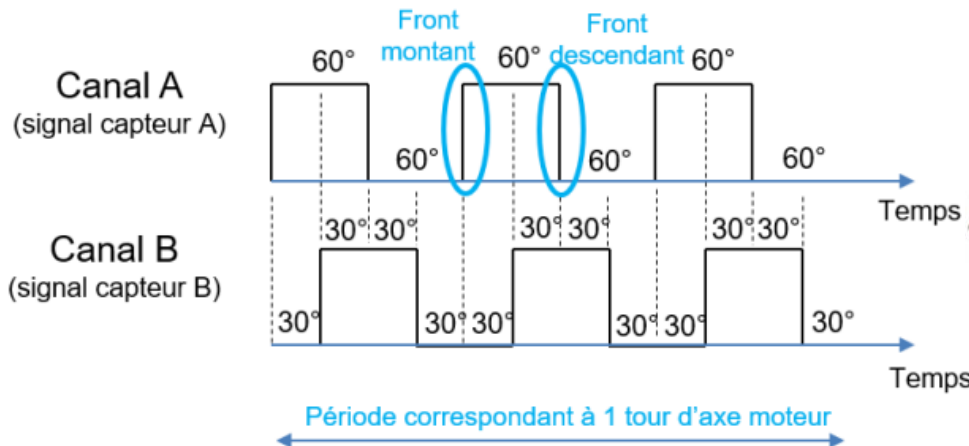
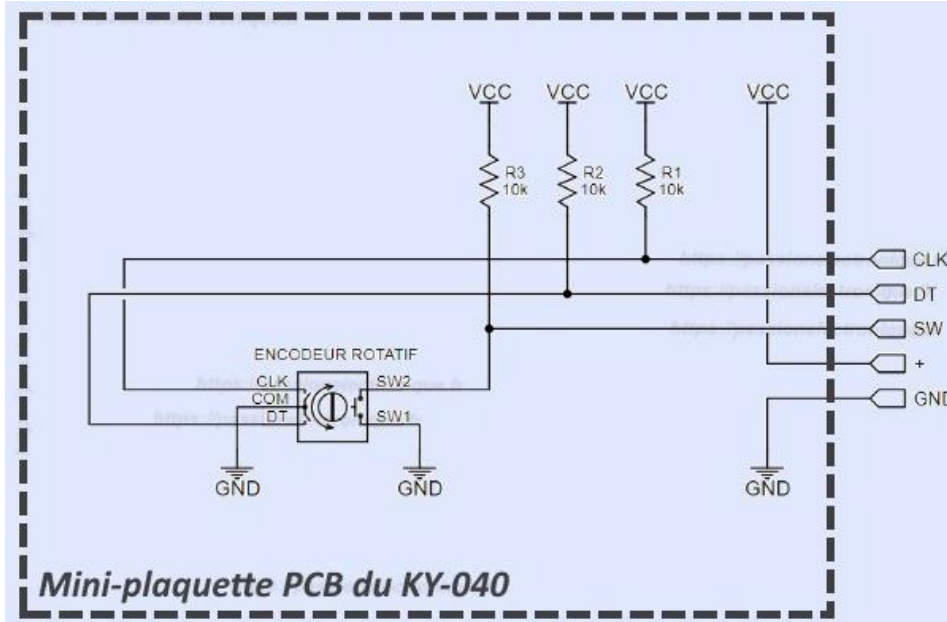




CERI

Programmation Pour l'Embarqué

Encodeur incrémental et interruptions : Contexte et Fonctionnement



L2 Informatique
UE S-E06-0207



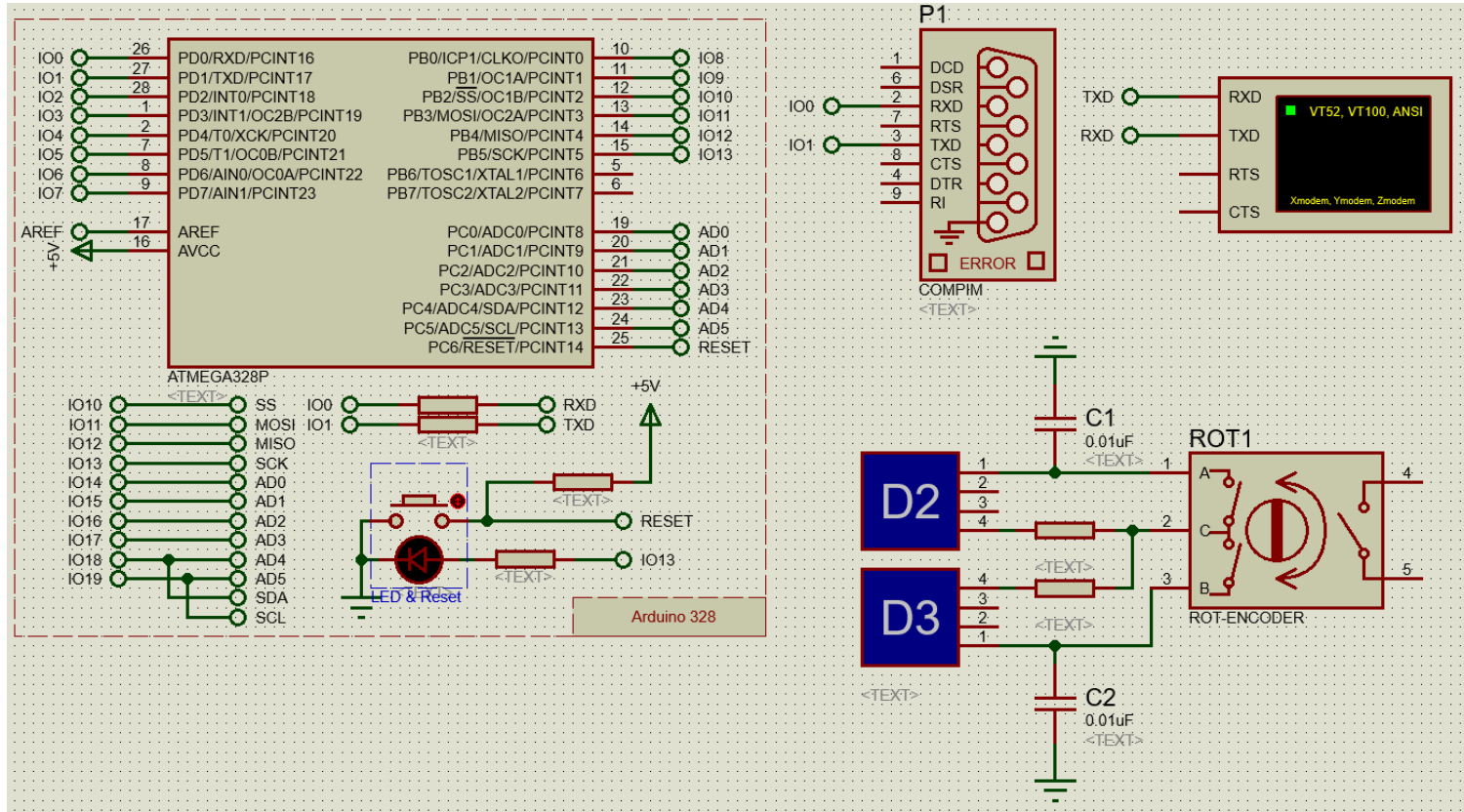
CERI

Programmation Pour l'Embarqué

Encodeur incrémental et interruptions : Schéma structurel

L2 Informatique

UE S-E06-0207



API Arduino:

API / fonction	Rôle	Syntaxe générique	Exemple
<code>attachInterrupt()</code>	Associer une fonction à une interruption externe	<code>attachInterrupt(broche, ISR, mode);</code>	<code>attachInterrupt(26, ISR_bouton, RISING);</code>
<code>detachInterrupt()</code>	Désactiver une interruption	<code>detachInterrupt(broche);</code>	<code>detachInterrupt(26);</code>
<code>digitalPinToInterrupt()</code>	Obtenir l'interruption associée à une broche	<code>digitalPinToInterrupt(broche)</code>	<code>digitalPinToInterrupt(26)</code>
RISING	Déclenchement sur front montant	<code>..., RISING);</code>	<code>attachInterrupt(26, ISR, RISING);</code>
FALLING	Déclenchement sur front descendant	<code>..., FALLING);</code>	<code>attachInterrupt(26, ISR, FALLING);</code>
CHANGE	Déclenchement sur changement d'état	<code>..., CHANGE);</code>	<code>attachInterrupt(26, ISR, CHANGE);</code>
LOW	Déclenchement lorsque l'entrée est à l'état bas	<code>..., LOW);</code>	<code>attachInterrupt(26, ISR, LOW);</code>
HIGH	Déclenchement lorsque l'entrée est à l'état haut	<code>..., HIGH);</code>	<code>attachInterrupt(26, ISR, HIGH);</code>
<code>volatile</code>	Variable partagée avec la fonction d'interruption	<code>volatile type variable;</code>	<code>volatile bool etat = false;</code>



Encodeur incrémental et interruptions : TD programme à compléter

```
const int encoderA = _____;
const int encoderB = _____;

volatile long position = _____;
static long oldPosition = _____;

void encoderISR()
{   if (digitalRead(encoderB) == HIGH)
    position++;           else    position--;           }
void setup()
{   pinMode(encoderA, _____);
    pinMode(encoderB, _____);

    attachInterrupt(_____, _____, _____);
    Serial.begin(_____, _____);
}

void loop()
{   if (position _____ oldPosition)
    {   oldPosition = _____;
        Serial.print("Position = ");
        Serial.println(_____, _____);
    }
}
```



Bibliothèques Arduino



Bibliothèques Arduino : Comment installer ?





Rôle de l'API Arduino ?

Avec l'API Arduino

```
// Faire clignoter la LED intégrée de l'Arduino UNO  
// Broche 13 = LED_BUILTIN
```

```
void setup()  
{  
    pinMode(LED_BUILTIN, OUTPUT);  
}  
  
void loop()  
{  
    digitalWrite(LED_BUILTIN, HIGH); // Allume la LED  
    delay(1000);                     // Attend 1 seconde  
  
    digitalWrite(LED_BUILTIN, LOW);  // Éteint la LED  
    delay(1000);                     // Attend 1 seconde}  
}
```



Rôle de l'API Arduino ?

Sans API : accès direct aux registres de l'ATmega328P

```
// Faire clignoter la LED de l'Arduino UNO
// Accès direct aux registres de l'ATmega328P
void setup()
{
    // Configure PB5 en sortie
    DDRB = DDRB | (1 << PB5);
}
void loop()
{
    // Met PB5 à 1 → LED allumée
    PORTB = PORTB | (1 << PB5);

    delay(1000);

    // Met PB5 à 0 → LED éteinte
    PORTB = PORTB & ~(1 << PB5);

    delay(1000);
}
```



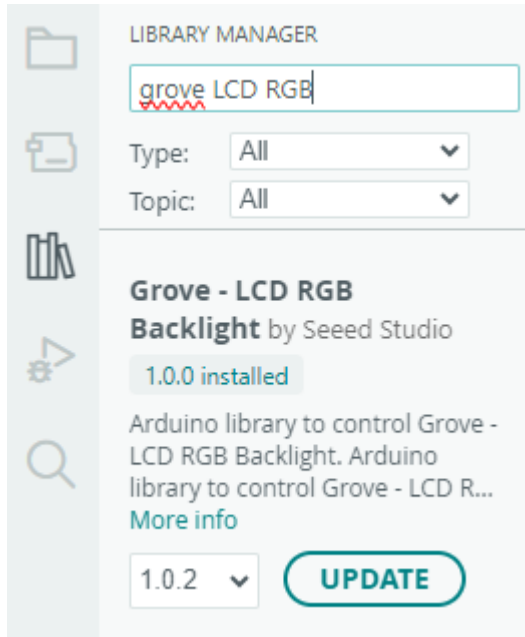
Rôle de l'API Arduino ?

Comparaison API / Registres

API Arduino	Registres
<code>pinMode(LED_BUILTIN, OUTPUT);</code>	<code>DDRB = DDRB (1 << PB5);</code>
<code>digitalWrite(LED_BUILTIN, HIGH);</code>	<code>PORTB = PORTB (1 << PB5);</code>
<code>digitalWrite(LED_BUILTIN, LOW);</code>	<code>PORTB = PORTB & ~(1 << PB5);</code>
Simple à comprendre	Nécessite de connaître l'architecture du microcontrôleur
Portable dans l'environnement Arduino	Dépend du microcontrôleur
Abstraction matérielle	Accès direct au matériel



Bibliothèques Arduino : Comment installer ? Exemple afficheur RGB LCD grove





Rôle d'une librairie programme générique

```
#include <MonComposant.h>
```

```
MonComposant monCapteur;
```

```
void setup()
```

```
{
```

```
    Serial.begin(9600);
```

```
    monCapteur.begin();
```

```
}
```

```
void loop()
```

```
{
```

```
    float valeur = monCapteur.getValue();
```

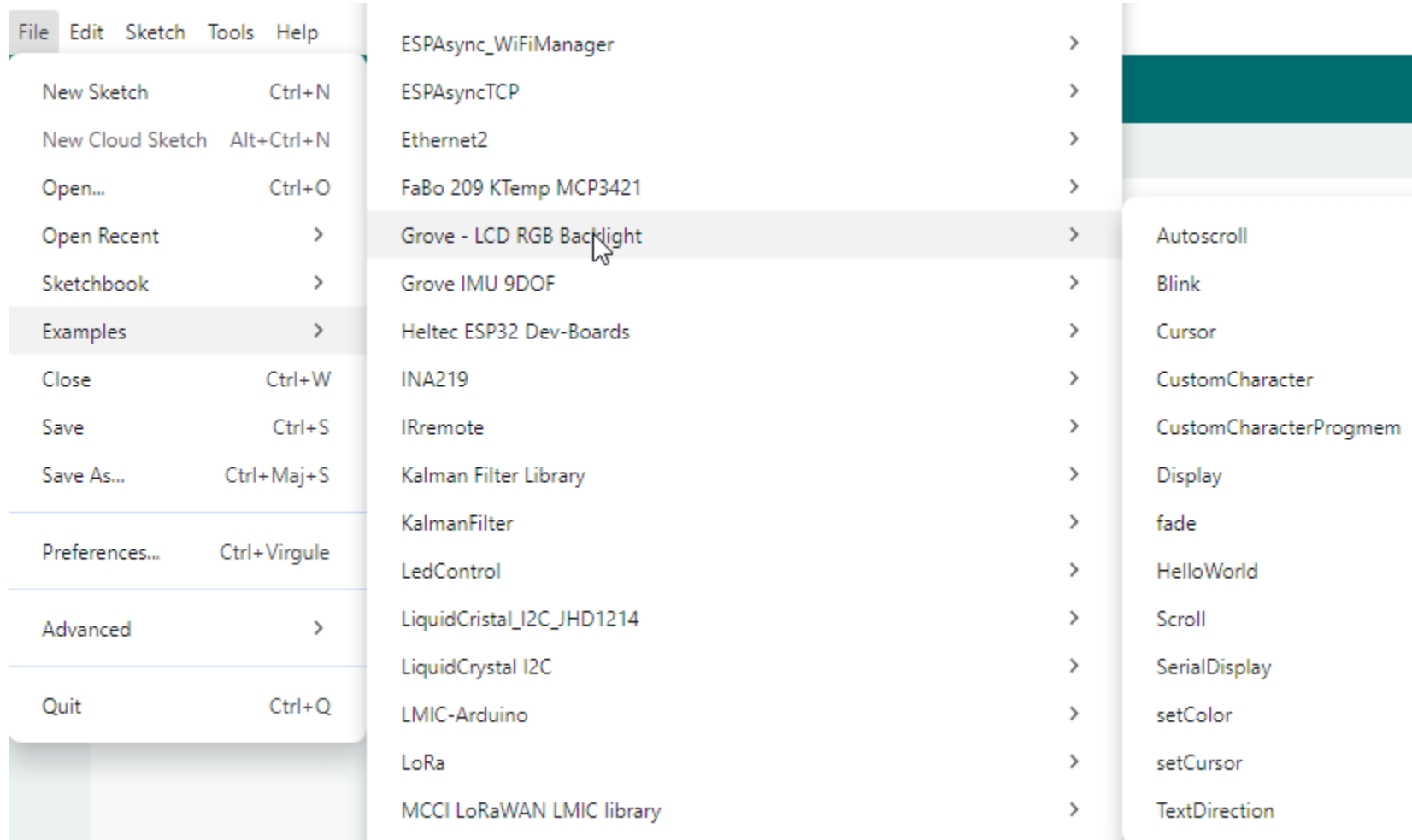
```
    Serial.println(valeur);
```

```
    delay(1000);
```

```
}
```



Bibliothèques Arduino : Utiliser les exemples fournis par le constructeur qui s'appuie sur l'API Arduino





TD : Compléter programme pour afficheur Grove RGB LCD

```
#include <Wire.h>
#include "rgb_lcd.h"
// Création de l'objet permettant de commander le LCD
// Nom de la bibliothèque nom donnée à l'objet

_____ lcd;

// Couleur du rétroéclairage : rouge
const int colorR = _____;
const int colorG = _____;
const int colorB = _____;

void setup()
{
    /*
    * Initialisation de l'afficheur.
    * Le premier paramètre correspond au nombre
    * de colonnes et le deuxième au nombre de lignes.
    */
    lcd.begin(_____, _____);
    lcd.setRGB(_____, _____, _____);

    _____

    /*
    lcd.print("_____");

    // Attente d'une seconde
    delay(_____);
}
```



Bibliothèques Arduino : Utiliser les exemples fournis par le constructeur qui s'appuie sur l'API Arduino

```
void loop()
{
    /*
    * Positionnement du curseur.
    * La numérotation commence à 0 :
    * colonne 0 = première colonne
    * ligne 0   = première ligne
    * ligne 1   = deuxième ligne
    */
    lcd.setCursor(_____, _____);

    /*
    * millis() renvoie le nombre de millisecondes
    * écoulées depuis le démarrage de la carte.
    * Pour obtenir une durée en secondes, il faut
    * convertir les millisecondes en secondes.
    */
    lcd.print(millis() / _____);

    // Petite temporisation avant la prochaine mise à jour
    delay(_____);
}
```



Bibliothèques Arduino : Utiliser les exemples fournis par le constructeur qui s'appuie sur l'API Arduino

```
#include <Wire.h>
#include "rgb_lcd.h"

rgb_lcd lcd;

const int colorR = 255;
const int colorG = 0;
const int colorB = 0;

void setup() {
  lcd.begin(16, 2);

  lcd.setRGB(colorR, colorG, colorB);
  lcd.print("hello, world!");

  delay(1000);
}

void loop() {
  lcd.setCursor(0, 1);
  lcd.print(millis() / 1000);

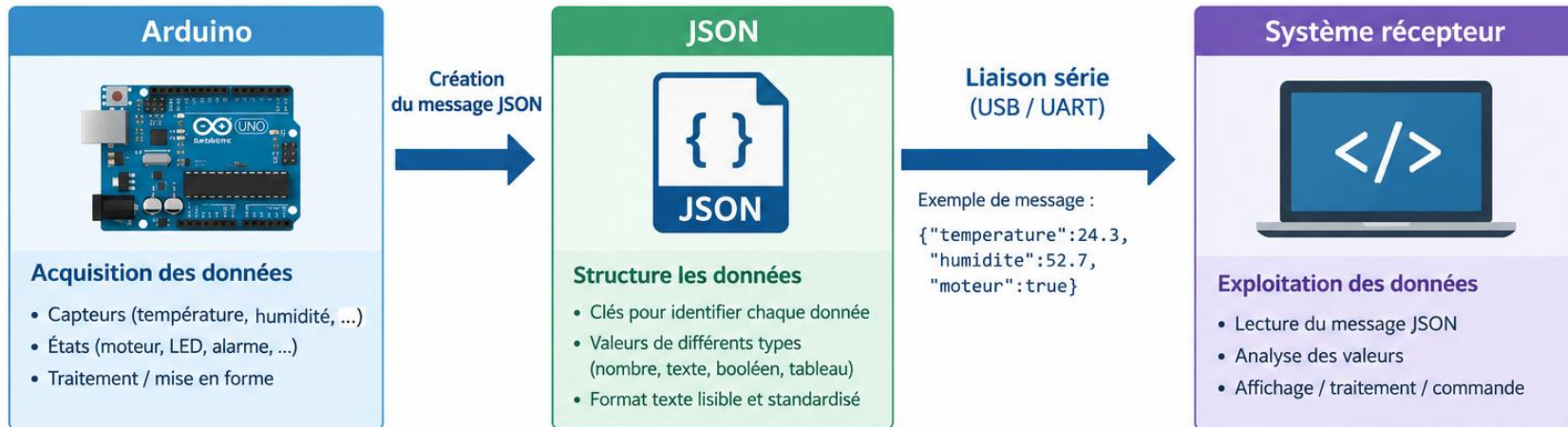
  delay(100);
}
```



Transmission des données serial au format JSON : Librairie **arduinoJSON**

Le rôle de JSON dans notre contexte

JSON permet de structurer les données échangées sur la liaison série entre Arduino et un système récepteur.



Pourquoi JSON ?

- ✓ **Lisibilité**
Les données sont compréhensibles par un humain.
- ✓ **Identification des données**
Chaque valeur est associée à une clé.
- ✓ **Interopérabilité**
Utilisé par de nombreux langages et outils.
- ✓ **Évolutivité**
Facile d'ajouter de nouvelles informations.

Exemple de message

```
{
  "temperature": 24.3,
  "humidite": 52.7,
  "vitesse": 850,
  "moteur": true,
  "alarme": false
}
```



Chaque clé permet d'identifier la donnée, quelle que soit sa position dans le message.

Structure d'un objet JSON

```
{ "clé" : valeur, ... }
```

- { }** **Objet**
Délimité par des accolades
- []** **Tableau**
Délimité par des crochets
- "clé"** **Clé**
Toujours entre guillemets
- valeur** **Valeur**
Nombre, texte, booléen, tableau...

En résumé





JSON le format d'écriture impose le type de la variable (PYTHON)

JSON	Signification	Python	Exemple JSON	Exemple Python
string	chaîne de caractères	str	"OK"	"OK"
number entier	nombre entier	int	850	850
number réel	nombre décimal	float	24.6	24.6
boolean	booléen	bool	true	True
boolean	booléen	bool	false	False
null	absence de valeur	NoneType	null	None
object	objet / structure clé-valeur	dict	{"id":12}	{"id":12}
array	tableau	list	[10,20,30]	[10,20,30]



JSON le format d'écriture impose le type de la variable (PYTHON)

Exemple

```
{
  "id": 12,                                # int
  "temperature": 24.6,                     # float
  "nom": "Moteur 1",                        # str
  "actif": True,                           # bool
  "default": False,                        # bool
  "erreur": None,                          # NoneType
  "vitesses": [500, 750, 1000],             # list
  "capteur": {                             # dict
    "type": "DHT11",                       # str
    "adresse": 4                           # int
  }
}
```



Librairie `#include <ArduinoJson.h>`

`#include`

`String messageRecu = "";`

`//=====`

`// SETUP : EMISSION D'UN MESSAGE JSON`

`//=====`

`void setup()`

`{`

`Serial._____;`

`//-----`

`// Création d'un document JSON permettant de construire le
message à envoyer.`

`//-----`

`JsonDocument doc;`

`//-----`

`// Création des différentes données du message.`

`//`

`// Le message doit contenir :`

`//`

`// "id" : 12`

`// "temperature" : 24.6`

`// "vitesse" : 850`

`// "moteur" : true`

`// "etat" : "OK"`

`//`

`// Compléter les cinq lignes suivantes.`

`//-----`

Tableau d'aide à la programmation

API / fonction	Rôle	Syntaxe générique	Exemple
JsonDocument	Créer un document JSON	JsonDocument nom;	JsonDocument doc;
doc["clé"] = valeur	Ajouter une donnée au JSON	document["clé"] = valeur;	doc["id"] = 12;
serializeJson()	Émettre le JSON	serializeJson(do cument, destination);	serializeJson(do c, Serial);
Serial.available ()	Vérifier si un message est reçu	Serial.available ()	if (Serial.availabl e())
Serial.readString Until()	Lire le message reçu	Serial.readString Until(caractère);	messageRecu = Serial.readString Until('\n');
deserializeJson()	Analyser le message JSON reçu	deserializeJson(document, texte);	deserializeJson(doc, messageRecu);
DeserializationE rror	Vérifier si le JSON reçu est valide	if (erreur)	if (erreur) { ... }
doc["clé"]	Lire une donnée du JSON	variable = document["clé"];	temperature = doc["temperature "];



Librairie `#include <ArduinoJson.h>`

`doc["id"] = _____;`

`doc["temperature"] = _____;`

`doc["vitesse"] = _____;`

`_____ = 850;`

`_____ = _____;`

`// Sérialisation du document JSON.`

`// Le document doit être envoyé directement sur le port
série.`

`serializeJson(doc, Serial);
Serial.println();`

`}`

`//=====`
`// LOOP : RECEPTION D'UN MESSAGE JSON`

`//=====`

`void loop()`

`{`



Librairie `#include <ArduinoJson.h>`

```
// -----  
// Vérifier si des caractères sont disponibles sur la  
liaison série.  
// -----  
if (Serial._____)  
{  
  // Lecture du message reçu jusqu'au caractère de fin de  
  ligne.  
  // La méthode readStringUntil() permet de lire les  
  caractères  
  // jusqu'à rencontrer le caractère '\n'.  
  messageRecu = _____;  
  // -----  
  // Affichage du message reçu.  
  // -----  
  Serial.print("Message reçu : ");  
  Serial.println(_____);  
  // -----  
  // Création d'un document JSON permettant de stocker  
  // les données reçues.  
  // -----  
  _____;  
  // -----  
  // Désérialisation du texte JSON.  
  // Stocker le résultat de deserializeJson() dans une  
variable  
  // appelée "erreur".  
  // -----  
  _____;  
  // -----  
  // Vérification de la validité du JSON reçu
```



Librairie `#include <ArduinoJson.h>`

```
// -----
// Vérification de la validité du JSON reçu.
// Si une erreur est détectée :
// - afficher "Erreur de JSON"
// - quitter le traitement avec return
// -----
```

```
if (_____)
{
  Serial.println("Erreur de JSON");
  return;
}
// -----
```

```
-----
// Récupération des différentes données du JSON.
// Le programme doit récupérer :
// id          → entier
// temperature → nombre réel
// vitesse     → entier
// moteur      → booléen
// etat        → chaîne de caractères
// -----
```

```
-----
int id = doc["id"];
float temperature = _____;
int vitesse = _____;
const char* etat = _____;
// -----
```



Librairie `#include <ArduinoJson.h>`

```
// -----  
// Vérification de la validité du JSON reçu.  
//  
// Si une erreur est détectée :  
// - afficher "Erreur de JSON"  
// - quitter le traitement avec return  
// -----
```

```
if (_____)  
{  
    Serial.println("Erreur de JSON");  
    return;  
}  
// -----
```

```
-----  
// Récupération des différentes données du JSON.  
//  
// Le programme doit récupérer :  
//  
// id          → entier  
// temperature → nombre réel  
// vitesse     → entier  
// moteur      → booléen  
// etat        → chaîne de caractères  
// -----
```

```
-----  
int id = doc["id"];  
float temperature = _____;  
int vitesse = _____;  
_____  
const char* etat = _____;  
// -----
```



Librairie `#include <ArduinoJson.h>`

```
// -----
// Vérification de la validité du JSON reçu.
// Si une erreur est détectée :
// - afficher "Erreur de JSON"
// - quitter le traitement avec return
// -----
if (_____)
{
  Serial.println("Erreur de JSON");
  return;
}
// -----
// Récupération des différentes données du JSON.
// Le programme doit récupérer :
// id                → entier
// temperature       → nombre réel
// vitesse           → entier
// moteur            → booléen
// etat               → chaîne de caractères
// -----
int id = doc["id"];
float temperature = _____;
int vitesse = _____;
_____ = _____;
const char* etat = _____;
// -----
// Affichage des données récupérées.
```



Librairie `#include <ArduinoJson.h>`

```
// -----  
// Créer un nouveau document JSON contenant la commande à  
transmettre.  
//  
// Le JSON devra avoir la forme :  
// {  
//   "id": 12,  
//   "vitesse": 500,  
//   "marche": true  
// }  
// -----  
JsonDocument commande;  
commande["id"] = id;  
commande["vitesse"] = _____;  
commande["marche"] = _____;  
// -----  
// Envoyer le document commande sur le port série.  
// -----  
_____  
Serial.println();  
}  
}
```



Librairie `#include <ArduinoJson.h>`

```

// -----
// Affichage des données récupérées.
// -----
Serial.print("ID : ");
Serial.println(______);
Serial.print("Temperature : ");
Serial.println(______);
Serial.print("Vitesse : ");
Serial.println(______);
Serial.print("Moteur : ");
Serial.println(______);
Serial.print("Etat : ");
Serial.println(______);
// -----
// Modifier la vitesse reçue.
// Si la vitesse reçue est inférieure à 500,
// imposer une vitesse minimale de 500.
// -----
if (______ < 500)
{
    _____ = 500;
}
// -----
// Créer un nouveau document JSON contenant la commande à
transmettre.
// Le JSON devra avoir la forme :
// {
//   "id": 12,
//   "vitesse": 500,
//   "marche": true,

```



Librairie `#include <ArduinoJson.h>` corrigé

```
#include <ArduinoJson.h>
String messageRecu = "";
// =====
// SETUP : EMISSION D'UN MESSAGE JSON
// =====
void setup()
{
    Serial.begin(115200);

    // -----
    // Création d'un document JSON permettant de construire le message
    // -----
    JsonDocument doc;
    // -----
    // Création des différentes données du message.
    // -----
    doc["id"] = 12;
    doc["temperature"] = 24.6;
    doc["vitesse"] = 850;
    doc["moteur"] = true;
    doc["etat"] = "OK";
    // -----
    // Sérialisation du document JSON.
    // -----
    // serializeJson() transforme le document JSON en texte.
    // Ici, le texte est directement envoyé sur le port série.
    // -----
    serializeJson(doc, Serial);
    Serial.println();
}
```



Librairie `#include <ArduinoJson.h>` corrigé

```
// =====  
// LOOP : RECEPTION D'UN MESSAGE JSON  
// =====  
void loop()  
{  
  // -----  
  // Vérification de la présence de données disponibles sur le p  
  // -----  
  if (Serial.available())  
  {  
    // -----  
    // Lecture du message reçu jusqu'au caractère '\n'.  
    // -----  
    // Dans le moniteur série, il faut donc sélectionner  
    // "Nouvelle ligne" comme fin de ligne.  
    // -----  
    messageRecu = Serial.readStringUntil('\n');  
    // -----  
    // Affichage du message reçu.  
    // -----  
    Serial.print("Message reçu : ");  
    Serial.println(messageRecu);  
  }  
}
```



Librairie `#include <ArduinoJson.h>` corrigé

```
// -----  
// Désérialisation du texte JSON.  
// deserializeJson() analyse le texte reçu et place les données  
// dans le document JSON.  
// -----  
DeserializationError erreur = deserializeJson(doc, messageReçu);  
// -----  
// Vérification de la validité du JSON.  
// -----  
if (erreur)  
{  
    Serial.println("Erreur de JSON");  
    return;  
}  
// -----  
// Récupération des différentes données du JSON.  
// -----  
int id = doc["id"];  
float température = doc["temperature"];  
int vitesse = doc["vitesse"];  
bool moteur = doc["moteur"];  
const char* etat = doc["état"];
```



```
// -----  
// Affichage des données récupérées.  
//  
Serial.print("ID : ");Serial.println(id);Serial.print("Température : ");  
Serial.println(temperature);Serial.print("Vitesse : ");Serial.print(vitesse);  
Serial.print("Moteur : ");Serial.println(moteur);Serial.print("Etat : ");  
Serial.println(etat);  
// -----  
// Vérification de la vitesse.  
//  
// Si la vitesse reçue est inférieure à 500 tr/min,  
// on impose une vitesse minimale de 500 tr/min.  
// -----  
if (vitesse < 500)  
{  
    vitesse = 500;  
}  
// -----  
// Création d'un nouveau document JSON.  
//  
JsonDocument commande;  
// -----  
// Construction du nouveau message JSON.  
// -----  
commande["id"] = id;  
commande["vitesse"] = vitesse;  
commande["marche"] = true;  
// -----  
// Sérialisation et émission du nouveau JSON.  
// -----  
serializeJson(commande, Serial);  
Serial.println();  
}
```